A prefix presentation of intuitionistic primitive recursive arithmetic

Four rule families, one recursive object syntax, and an antecedent-free alternative

Research prepared for Vladimir Reshetnikov

November 11, 2026

Abstract

The current ConciseIPRA development already has four instruction families with a fixed reflexivity clause. We retain its arithmetic and intuitionistic logical strength and simplify its presentation: every native Eq/D formula is a chain of paired binders ending in one equation, so it can be represented by a natural number and two terms. Only the numerical term syntax remains recursive. Substitution in a formula becomes substitution in its two terms at a depth determined by that number. We give the complete calculus, an executable checker, explicit translations to and from the current calculus, and the resulting whole-domain bi-interpretation with iPRA.

A separate mathematical result removes the antecedent field of every context clause. An asymmetric iterator gate represents ordinary assumptions as homogeneous conditional closures and has a literally reflexive zero instance. This alternative is proved equivalent, but its measured checker is larger than the corresponding prefix checker with paired clauses. The package distinguishes these results from a claim of global minimality: it does not beat the repository's most compressed existing implementations.

1 Result, comparison target, and limits

The inspected baseline is commit

`17898ea7e6a1300dfdcad71040d5daeed67916d4`

of *ProveIt*, under `Logic/PrimitiveRecursiveArithmetic/ConciseIPRA` [1]. All comparisons in this report refer to that pinned snapshot. The baseline already contains the clause calculi `CPrf`, `KPrf`, and `VPrf`. In particular, putting reflexivity into a fixed clause, using two-case computational equality, and using four instruction families are inherited results [2, 3, 4]. They are not claimed as new here.

The primary presentation $\mathcal{F}$ makes three precise choices:

1. A formula is $\langle k; s = t\rangle$: a binder count and a pair of numerical terms. There is no recursive formula constructor in the certificate grammar.

2. A clause has one antecedent and one unary consequent template. A reflexive-leaf antecedent may be discharged without a supplied proof. This includes the fixed reflexivity clause and ordinary assumptions.

3. There are four proof constructors: Use, Eval, Elim, and Intro. Elim synthesizes its conclusion and checks that its local witness has not escaped.

The first choice is the main representational reduction. The second and third make the complete interface uniform. Section 6 separates the complete comparison from the comparison in which the control uses the same proof interface.

The arithmetic basis itself is unchanged: a scalar iterator has a seed and a unary update, and the restricted induction mechanism is the inherited witness-directed introduction. The new claim is a smaller explicit presentation and implementation in the reported comparison, not a new strength theorem about arithmetic or a minimum over all programming languages and proof encodings. The existing compact checkers remain smaller in absolute source size. The additional homogeneous-context theorem is a mathematical simplification whose implementation cost is reported rather than omitted.

2 The complete prefix calculus

2.1 Numerical terms and formula prefixes

There is one numerical sort. Terms are

$$t ::= x \mid 0 \mid St \mid \mathsf{I}(n, a; u.h),$$

where only u is bound in h. A term may contain any finite nesting of iterators; there are no function variables or universal evaluator.

A formula is the finite record $\langle k;\ s = t\rangle$, with $k \in \mathbb{N}$. It binds k ordered pairs of numerical variables in s, t. Its expansion is

$$\mathrm{unpack}\langle k;\ s = t\rangle = \mathsf{D}_{x_1,y_1} \cdots \mathsf{D}_{x_k,y_k}(s = t). \tag{1}$$

Thus $\langle 0;\ s = t\rangle$ is an equation. The names in (1) are explanatory; the checker uses bound indices. The binder count is syntactic data, not an object-level numerical variable quantified by the theory.

For readability, $\mathsf{D}_{x,y}B$ below denotes adding one pair to a prefix. It is an operation on the record, not an additional recursive syntax. $B(a, b)$ denotes capture-free opening of its two designated external slots. A unary template $B(y)$ has one designated external slot. All other free parameters remain fixed.

The intended ordinary reading of the prefix is recursively specified by

$$(s = t)^\circ = (s = t), \qquad (\mathsf{D}_{x,y}B)^\circ = \exists x(B^\circ(x, 0) \to \forall y\, B^\circ(x, y)). \tag{2}$$

This recursion defines a translation of each finite syntactic object. It does not assert the existence of an internal truth predicate. The designated contradiction is $\langle 0;\ 0 = S0\rangle$.

2.2 Contexts and their one fixed clause

A context is a finite reusable list of clauses $[A; y.B(y)]$. Its ordinary reading is $A^\circ \to \forall y B^\circ(y)$; y is not free in A. Each proof begins with the clause

$$\rho = [\,0 = 0;\ \ y.y = y\,]. \tag{3}$$

In the closed-certificate protocol this is the sole initial entry, and the list grows only through Intro and Elim below. For mathematical statements about sequents, additional well-scoped clauses may be retained as external assumptions; the delivered verifier checks closed certificates. An ordinary local assumption A is the constant clause $[0 = 0; v.A]$, where v is unused. There is no separate hypothesis rule.

Call $\langle k;\, r = r\rangle$ *reflexive at its leaf*. The two term trees must be literally identical. No numerical evaluation, provability test, or normalization is used in this test. Section 3.1 proves that every such formula is a theorem, including $k > 0$.

2.3 Four primitive rule families

Write $\Gamma \vdash_{\mathcal{F}} A$ for derivability. Contexts always include ρ. The entire primitive inventory is as follows.

Use. For any retained clause $[A; y.B] \in \Gamma$ and any term t,

$$\frac{\Gamma \vdash_{\mathcal{F}} A}{\Gamma \vdash_{\mathcal{F}} B(t)}. \tag{Use}$$

The minor is omitted when A is reflexive at its leaf. Otherwise one minor derivation is required. Clause use does not consume the clause.

Eval. From two equations,

$$\frac{\Gamma \vdash_{\mathcal{F}} s = t \qquad \Gamma \vdash_{\mathcal{F}} b = r}{\Gamma \vdash_{\mathcal{F}} r = \Phi(s,t,b)}, \tag{Eval}$$

where Φ is defined in exactly two cases:

$$\Phi(s,t,b) = \begin{cases} t, & b \equiv \mathsf{I}(0, s; u.h), \\ h[t/u], & b \equiv \mathsf{I}(Sc, a; u.h) \text{ and } s \equiv \mathsf{I}(c, a; u.h). \end{cases} \tag{4}$$

All displayed matches are literal syntactic matches, with the usual capture-free interpretation of a binder. There is no extra equality-join case. The two lines of (4) are the inherited computational equality interface [4, 2].

Elim.

$$\frac{\Gamma \vdash_{\mathcal{F}} \mathsf{D}_{x,y} B(x,y) \qquad \Gamma, [B(b,0); y.B(b,y)] \vdash_{\mathcal{F}} A}{\Gamma \vdash_{\mathcal{F}} A}. \tag{Elim}$$

The numerical parameter b is fresh for Γ, the conclusion A, and the fixed parameters of the eliminated formula. The checker enforces this by assigning a new free level and validating the branch result in the outer scope, including occurrences in unused iterator bodies.

Intro. For z fresh for Γ, t, and the conclusion's fixed parameters,

$$\frac{\Gamma, [0 = 0; v.B(t,\eta)] \vdash_{\mathcal{F}} B(t, Sz)}{\Gamma \vdash_{\mathcal{F}} \mathsf{D}_{x,y} B(x,y)}, \qquad \eta = \begin{cases} z, & B \text{ is an equation and } t \equiv 0, \\ 0, & \text{otherwise.} \end{cases} \tag{Intro}$$

Here v is unused. In the prefix representation the test "B is an equation" means that the conclusion has binder count exactly 1. This is the established single witness-directed introduction; its case at a witness that is not literally zero supplies numerical cases, and its atomic zero-witness case supplies equation induction [1, 5].

There are no other primitive logical axioms, equality axioms, computation axioms, induction schemas, structural rules, or inference rules in $\mathcal{F}$. The fixed clause (3), the two syntactic cases of Eval, and the stated side conditions are part of this complete specification.

3 Why the reduced interface is sufficient

3.1 Reflexive leaves and ordinary assumptions

Lemma 3.1 (Reflexivity)**.** *For every well-scoped term t, $\Gamma \vdash_{\mathcal{F}} t = t$.*

Proof. Use ρ at t. Its antecedent is the literally reflexive equation $0 = 0$, so the minor is omitted. □

Lemma 3.2 (Reflexive prefixes)**.** *Every well-scoped formula $\langle k; r = r\rangle$ is derivable in every context. The same statement holds in the inherited `CPrf` and `VPrf` calculi after unpacking. The derivations can be constructed effectively from the formula, independently in each calculus.*

Proof. Induct on k. The case $k = 0$ is the previous lemma in $\mathcal{F}$, primitive Refl in `CPrf`, and use of the unconditional fixed reflexivity clause in `VPrf`. If $k > 0$, introduce the outer pair with witness $S0$. This is case mode regardless of the matrix's root. The target at the fresh successor has binder count $k - 1$, and its two leaf terms are still identical after opening the witness and counter slots. Apply the induction hypothesis to this target and ignore the added clause. Close Intro. Consequently the optional minor in Use only abbreviates a derivation already available in the system. □

Use of the constant clause $[0 = 0; v.A]$ gives A at any target, again without a minor. Thus the ordinary-assumption interface is available with no separate hypothesis constructor.

3.2 Computation and equality

Both iterator computation equations are Eval on two reflexivities:

$$\mathsf{I}(0, a; u.h) = a, \qquad \mathsf{I}(Sn, a; u.h) = h[\mathsf{I}(n, a; u.h)/u]. \tag{5}$$

To see that no equality principle has disappeared, let $F(s) = \mathsf{I}(0, s; u.u)$. From $s = r$ and reflexivity of $F(s)$, the zero case of Eval gives $F(s) = r$. Applying the same zero case to $s = t$ and this last equation gives

$$s = t,\ s = r \ \vdash_{\mathcal{F}}\ r = t. \tag{6}$$

Taking one premise reflexive yields symmetry, and symmetry plus (6) yields transitivity.

For an arbitrary term template $h(u)$ and a premise $s = t$, the zero case gives $\mathsf{I}(0, s; u.h) = t$. The successor case, with reflexivity of $\mathsf{I}(S0, s; u.h)$, gives $\mathsf{I}(S0, s; u.h) = h(t)$. Independently, zero Eval on $s = s$ and reflexivity of $\mathsf{I}(0, s; u.h)$ gives $\mathsf{I}(0, s; u.h) = s$. Successor Eval with reflexivity of $\mathsf{I}(S0, s; u.h)$ then gives $\mathsf{I}(S0, s; u.h) = h(s)$. Joining these two equations yields $h(s) = h(t)$, without first rewriting under h. This supplies arbitrary numerical congruence and the inherited transport rule.

Full formula transport then follows by induction on the number of D-blocks: eliminate the source paired formula, introduce the target at the fresh witness, convert its local base, use the retained clause at the successor, and convert back. The fresh witness forces case mode. All matrix conversions are at smaller binder count.

3.3 Induction is present and remains restricted

Let $E(y)$ be a literal equation. Given proofs of $E(0)$ and of $E(Sz)$ from the local assumption $E(z)$, Intro at the unused witness 0 gives $\mathsf{D}_{x,y}E(y)$. Elim exports $[E(0); y.E(y)]$; Use with the base proof gives $E(t)$ for any term t. This is contextual equation induction. It applies to equations, not to arbitrary formulas.

The inherited witness-rule equivalence also recovers the general case introduction. Its exceptional atomic case at witness 0 can use the syntactically nonzero but provably zero term $\mathsf{I}(0,0;u.0)$ and the derived transport. This explains why the literal witness test is harmless, while also explaining why substitution admissibility is proved through the equivalence and cannot be read off from an invariant witness test alone.

4 Exact correspondence with the current calculus

Lemma 4.1 (Unique prefix decomposition)**.** *Every formula in the current Eq/D grammar has a unique representation $\langle k;\, s = t\rangle$, up to the ordinary representation of bound variables. Packing and unpacking are inverse operations on these finite trees.*

Proof. An equation supplies $k = 0$. At a D node there is exactly one formula child; strip that node and add one to the child's count. The grammar has no branching formula constructor. Repeatedly stripping D therefore reaches exactly one equality leaf. Reconstruction adds exactly the stripped nodes in their original order. □

With de Bruijn indices starting at 1, the two outer slots of the matrix of a rank-$k+1$ formula have indices $2k+2$ and $2k+1$ at its equality leaf. Its k inner pairs occupy the smaller positive indices. All concrete substitution values are closed with respect to exterior positive indices; their free numerical parameters have negative levels. Opening the outer witness substitutes at $2k+2$; after that, opening the outer counter substitutes at $2k+1$. An additional iterator binder increments the index being sought by one. This gives the following commuting law.

Lemma 4.2 (Opening commutes with packing)**.** *Packing a capture-free opening of a nested formula equals opening its packed record by the preceding index rule. The same holds under every iterator binder and for simultaneous choices of the external slots.*

Proof. Strip the formula prefix. Each stripped pair adds exactly two to the substitution depth. At the equality leaf, induct on its two numerical terms. A variable is replaced exactly at the indicated index; larger indices decrease by one when that slot is removed. Successor adds no binder, and the update body of an iterator adds one. These are exactly the clauses of capture-free substitution on the nested representation. □

Theorem 4.3 (Equivalence of finite derivations)**.** *The prefix calculus $\mathcal{F}$ and the Eq/D (that is, Q-free) part of the pinned native clause calculus derive the same sequents after packing or unpacking formulas and clauses. The translations of derivations are effective. Both formula round trips are syntactic identities, not merely agreement in the standard model.*

Proof. First compare with `CPrf`, whose contexts already consist of pairs $[A; y.B]$ and whose ordinary assumptions are $[0 = 0; v.A]$.

From `CPrf` to $\mathcal{F}$, pack all formulas and add the fixed clause ρ to the context. Primitive reflexivity becomes a zero-premise Use of ρ. The two Eval patterns, ordinary Use, Elim and the witness-directed Intro otherwise translate directly. Opening commutes with packing, and the sets of free numerical parameters are unchanged, so all substitutions and eigenvariable conditions agree. An induction on the source derivation preserves the extra fixed clause under every scope extension.

For the reverse direction, carry the context invariant that every retained clause is either the distinguished ρ or the packing of a `CPrf` context clause. Use of the distinguished ρ becomes `CPrf` reflexivity. Any other Use carries its translated antecedent derivation; if that derivation was omitted, supply the independently proved `CPrf` reflexive-prefix theorem instead. Then apply the ordinary

`CPrf` Use rule. Eval, Elim and Intro translate directly and preserve this invariant. Thus no unproved clause-cut or context-extension principle is required.

The pinned theorem `vprf_iff_cprf` gives the corresponding four-family `VPrf` formulation. Its empty antecedent is represented by $0 = 0$, and its singleton antecedent remains the same formula. The above proof explains exactly how the fixed reflexivity clause and all skipped minors are handled; it does not assume identical certificate grammars. Synthesizing Elim's conclusion instead of storing another copy preserves its outer-scope side condition. Finally, the exact native equivalences of `CPrf`/`VPrf` are the pinned inherited results cited in the report. □

5 Ordinary logic and the bi-interpretation with iPRA

The preceding theorem is an equivalence of complete proof presentations. Here iPRA means intuitionistic first-order logic with equality, symbols for every finite primitive-recursive description and their defining equations, successor separation and injectivity, and contextual induction for quantifier-free formulas. We specify the arithmetic interpretation as well, so that the conclusion is stronger than equiconsistency or standard-model representability.

5.1 All ordinary logical operations are available

The logical macros below are inherited from the native construction. They can be packed into prefix records after expansion. Put $C(v, a, b) = \mathsf{I}(v, a; u.b)$ with u unused. Define

$$G_v(s = t) = (C(v, 0, s) = C(v, 0, t)), \qquad G_v(\mathsf{D}B) = \mathsf{D}(G_v B).$$

The native calculus proves $G_0 A$ and $G_{Sv} A \dashv\vdash A$. To select between formulas of different depths, pad the shallower one with $A \dashv\vdash \mathsf{D}_{u,y} G_y A$, using an unused witness u. After their depths are aligned, define the selector at the leaf by

$$M_v(s = t, a = b) = (C(v, s, a) = C(v, t, b)),$$

and pass it through aligned D-blocks. Its zero and positive endpoints are A and B, respectively. Let $\delta(v) = C(v, 1, 0)$ and $p(z) = \mathsf{I}(z, 0; u.\delta(u))$. With all displayed dummy variables fresh, the macros are

$$\begin{aligned} A \Rightarrow_* B &= \mathsf{D}_{u,z} M_z(A, B), \\ A \wedge_* B &= \mathsf{D}_{u,z} G_z M_{p(z)}(A, B), \\ \exists_* x\, A(x) &= \mathsf{D}_{x,y} G_y A(x), \\ A \vee_* B &= \exists_* x\, M_x(A, B), \\ \forall_* x\, A(x) &= A(0) \wedge_* \mathsf{D}_{u,z} G_z A(z), \\ \bot_* &= (0 = S0). \end{aligned} \tag{7}$$

The gate, selector, and paired introduction/elimination prove the usual intuitionistic introduction and elimination rules. Numerical cases derive $x = 0 \vee_* \exists_* v\, x = Sv$. Under $0 = S0$, selector computation and transport give $s = C(0, s, t) = C(S0, s, t) = t$, and induction on formula depth gives explosion for every native formula. The inherited detailed proof is included as `reference/inherited_choice_basis.tex`; the corresponding Lean results are identified in [1, 6].

5.2 The ordinary scalar realization

Let T_{eq} be intuitionistic first-order logic with actual equality over the scalar iterator language, with equations (5), contextual induction for literal equations, and the sentences

$$\forall x\,(x = 0 \vee \exists y\, x = Sy), \qquad (0 = S0) \to \bot. \tag{8}$$

These are the ordinary counterpart of the derived native interface. The explicit falsity bridge is essential in this ordinary description: positive equations alone would admit a one-element structure. The prefix-to-ordinary map is (1)–(2), and an ordinary formula is mapped back by (7) followed by packing. Native completeness and both formula round trips are proved in the pinned development. Packing adds an exact syntactic isomorphism, so those results transfer to $\mathcal{F}$.

5.3 The numerical interpretations

Interpret $\mathsf{I}(n, a; u.h)$ in iPRA as the primitive recursion with initial value a and update h, ignoring the recursion counter in that update. Each fixed nested term gives a finite primitive-recursive description. The interpretation domain is the entire numerical domain, and its equality is actual equality.

Conversely, the scalar theory first derives a predecessor P with $P0 = 0$ and $P(Sn) = n$, then a numerical test reflecting actual equality, then quantifier-free induction, and then pairing and projections. This order avoids assuming pairing or an equality detector in the predecessor argument. The inherited detailed proofs are supplied in the reference directory [7]. If π, L, H satisfy $L(\pi(a, b)) = a$ and $H(\pi(a, b)) = b$, a fixed primitive recursion

$$f(0, \vec{p}) = g(\vec{p}), \qquad f(Sn, \vec{p}) = h(n, f(n, \vec{p}), \vec{p})$$

is compiled to

$$\begin{aligned} V(n, \vec{p}) &= \mathsf{I}\big(n, \pi(0, g^*(\vec{p})); \, u.\pi(SL(u), h^*(L(u), H(u), \vec{p}))\big), \\ f^*(n, \vec{p}) &= H(V(n, \vec{p})). \end{aligned} \tag{9}$$

Equation induction proves $L(V(n, \vec{p})) = n$, and the two projection identities prove the required recursion equations. Pairing stores a finite state; it does not replace the numerical universe by a range of codes.

Theorem 5.1 (Whole-domain bi-interpretability and equiconsistency)**.** *The ordinary realization of $\mathcal{F}$ is bi-interpretable with full first-order intuitionistic primitive recursive arithmetic. Both numerical domains are whole, both equality relations are actual equality, and both composite isomorphisms have graph $x = y$. The two theories are equiconsistent.*

Proof. Compose the exact prefix/native correspondence with the pinned native–T_{eq} interpretation and the scalar–iPRA interpretation. For completeness, the numerical round-trip argument is as follows. External induction on each fixed scalar description shows that its double translation satisfies its original recursion equations. Equation induction, with the seed and passive parameters held fixed and only afterward instantiated, proves uniqueness of two iterators with the same seed and update. For each fixed ordinary primitive-recursive description, external induction on that description and internal uniqueness of primitive recursion give the same conclusion. Hence each composite fixes every numerical operation provably, at arbitrary internal arguments, and $x = y$ is the required internally definable isomorphism. Formula round trips follow from the native logical macro theorem and the inverse prefix translations. Contradiction is preserved in both directions by (8) and $\bot_* = 0 = S0$. Effective finite proof translations give equiconsistency. This does not claim that either theory proves its own consistency. □

6 Executable specification and measurement

6.1 JSON syntax and binding discipline

The standalone checker uses only the Python standard library. Its numerical terms are encoded by

Mathematical object	JSON encoding
Zero	`0`
Bound variable, index $i > 0$	integer i
Free eigenparameter, level $i < 0$	integer i
Successor St	`[t]`
Iterator $\mathsf{I}(n, a; u.h)$	`[n,a,h]`
Formula $\langle k;\ s = t\rangle$	`[k,s,t]`

The context determines whether a three-element array is a formula record or an iterator term. The checker validates each use at its required sort. Booleans and floating-point numbers are not numerical indices.

At outer scope n and binder depth d, a numerical integer is admissible exactly when $-n \le i \le d$. Only an iterator's update adds one to d. A formula of count k validates its leaf terms at depth $2k$. The fresh parameter introduced by a local branch is $-n - 1$. All branches add exactly one context entry, so the implementation recovers n as the context length minus the initial entry.

Substitution inserts only terms that are closed with respect to exterior positive indices. Such inserted terms may have their own internal binders, but they contain no positive index referring to an ambient binder. Consequently the insertion itself needs no extra lifting; the traversal still increments the sought index under an iterator binder and decreases larger indices when an external slot is removed. This invariant is central to the short implementation and is checked at every certificate-supplied witness. The homogeneous alternative additionally needs lifting when it places an existing leaf term under a new iterator binder.

Proof nodes are

```
["u", clause_index, target_term, minor_proof]
["v", major_equation_proof, minor_equation_proof]
["e", paired_formula_proof, branch_proof]
["i", paired_goal_formula, witness_term, branch_proof]
```

Input is exactly `{"goal": F, "proof": P}`. In a Use node whose antecedent is reflexive at its leaf, `minor_proof` is an inert JSON field; the examples use `[]`. Its contents are not a claimed subderivation. Every active premise, supplied term, introduced formula, and elimination result is checked. The public goal must be a closed well-scoped formula. The program prints `ACCEPT` on success and `REJECT` on a detected exception; finite tests do not replace the soundness proof. As with the comparison programs, completeness refers to the ideal finite-derivation algorithm; a particular Python process can exhaust its recursion, parsing, or memory limits on large inputs. The binder count may encode a long finite prefix succinctly.

6.2 Measured comparisons

Measurements use CPython 3.12.14. Genuine module, function, and class docstrings are removed, the complete module is normalized by `ast.unparse` with one terminal newline, and significant Python tokens, all AST nodes, and UTF-8 bytes are counted. Layout, comments, encoding markers, and end markers are not significant tokens. Gzip level 9 with a fixed zero timestamp is informational.

No gate, scope checker, parser interface, or active inference routine is omitted from a measured module.

Complete implementation	Tokens	AST nodes	Bytes	Gzip-9
Existing `uniform_clauses`	807	931	2912	994
Matched nested control	791	890	2763	931
Prefix calculus $\mathcal{F}$	**788**	900	**2700**	941
Homogeneous prefix variant	840	965	2843	979

Relative to the exact existing four-family program, the delivered prefix checker saves 19 tokens, 31 AST nodes, and 212 normalized bytes (approximately 2.35%, 3.33%, and 7.28%, respectively). This comparison includes the simplified clause and certificate interfaces as well as the prefix representation.

The matched nested control uses the same four proof tags, synthesized elimination result, single-antecedent clauses, implicit scope protocol, reflexive-leaf shortcut, and numerical term codec. It retains nested Eq/D formulas. Against this control, the prefix representation alone saves 3 tokens and 63 bytes (approximately 2.28% of bytes), while adding 10 AST nodes and 10 gzip bytes. Thus it is not a reduction in every coordinate against that tighter control.

The antecedent-free implementation in the last row pays for its gate and capture-free lifting. Its structural theorem does not yield the smallest checker in this table. The counts also do not establish a new repository-wide minimum: the existing validated native implementation has 788 *bytes*, and a separate compact implementation has 516 bytes under its canonical public-goal contract. Those frozen sources are included for transparency. The number 788 in the prefix row is a *token count*; the prefix program has 2700 normalized bytes. These distinct units and contracts must not be conflated.

The exact sources, source hashes, normalized copies, compressed copies, and measurement procedure are in `code/` and `code/evidence/checker_metrics.json`.

6.3 Verification and limits of the executable evidence

The main reproduction run constructed 37 positive certificates and translated each to the matched nested checker and the independently retrieved existing four-family checker. Each ran in all three programs under both ordinary Python and `python -O`. An additional 22 negative inputs ran in both modes against the prefix checker. This is 266 subprocess expectations, with zero mismatches. The positive cases include zero and successor computation, derived symmetry, nested paired binders, and a nontrivial equation-induction proof of

$$\mathsf{I}(y, 0; u.Su) = y$$

followed by specialization. The negative cases include incorrect induction steps, escaped eigenparameters, malformed or out-of-scope terms, invalid clause indices, nonintegral binder counts, and compound premises used as equations.

There were also 2001 checks that opening commutes with expansion to the nested representation. A separately written audit used environment-based numerical semantics, rather than another syntactic substitution routine, for 400 matrix openings, 400 witness openings, and 300 iterator-body substitutions. It passed 20 additional rejection cases and 5 accepted examples. The scope attacks include a forbidden parameter beneath a dead zero-iterator body and beneath a formula binder; dead code is still checked for scope.

The deterministic generators, concrete certificates, full results, and source hashes are included. The independent semantic checks use bounded samples and a finite test interpretation for formula-opening comparisons; they are not asserted to be a model of the entire arithmetic theory. Written

derivation translations supply the metatheorems, while these checks supply evidence that the implementations follow them.

From the package directory, run

```
python3 code/reproduce_checker.py
python3 code/audit_flat.py code/flat_control.py
python3 code/flat_control.py < code/examples/equation_induction.json
```

The last command checks the supplied induction certificate. The example filenames and complete format description are also listed in `README.txt` and `code/FORMAT.txt`.

The correctness results in this report are written mathematical proofs. The inherited native equivalences and arithmetic bi-interpretation have Lean formalizations in the cited repository. This package does not claim a newly checked Lean or Rocq formalization of its prefix checker or of the homogeneous-context theorem. The Python programs verify certificates for the presented inference rules; they are not proof assistants checking the metatheory automatically.

7 Further research questions

1. Can the prefix representation reduce the smallest fully validated compact checker while preserving that checker's exact public language and successful-trace invariant? The matched readable reduction does not settle this question.

2. Can homogeneous contexts be implemented without paying for an additional gate operation, while keeping all compound assumptions and their exact conditional meaning? The theorem in Appendix A shows that the antecedent field is mathematically eliminable, but the measured implementation still pays more for that elimination.

3. Can a substitution-stable introduction, such as the existing diagonal introduction, be paired with the prefix interface at lower total checker cost than the witness-directed rule? Formula depth makes the atomic test trivial, but it does not eliminate the arithmetic guard.

4. Can the prefix packing and opening lemmas, followed by the homogeneous gate lemma, be added as short modules over the existing Lean `CPrf`/`VPrf` development? This would check the new metatheory without redoing the inherited arithmetic bridge.

5. What minimum is attainable under a fixed, explicit certificate contract? A useful comparison should fix goal grammar validation, scope checking, accepted padding, normalization, and the source measurement procedure before optimizing implementations.

A The homogeneous four-family calculus

The proposed calculus retains the scalar terms and the original ordinary meaning of the repository's Eq/D language. The change is in retained assumptions: every context entry is a single one-variable formula template. No entry has a separately stored antecedent, a list of antecedents, or a flag distinguishing facts from conditional closures.

A.1 Language, ordinary reading, and the gate

The term language is

$$t ::= x \mid 0 \mid St \mid I(n, a; u.h),$$

where u is bound only in h. Formulas are

$$A ::= (s = t) \mid D_{x,y}A.$$

The ordinary reading is unchanged:

$$D_{x,y}B(x, y) \;:=\; \exists x \,(B(x, 0) \to \forall y\, B(x, y)).$$

Write Q_yB as a metalinguistic abbreviation for $B(0) \to \forall yB(y)$, not as a constructor of the new formula language. A context Δ is a finite reusable list of templates $B(y)$; its ordinary reading is the list of formulas Q_yB. The distinguished initial entry is

$$\rho(y) \equiv (y = y).$$

It remains present in every context. Its ordinary reading is a theorem.

For a fresh variable v, define

$$\kappa_v(s = t) := \big(I(0, s; u.t) = I(v, s; u.t)\big), \qquad \kappa_v(D_{x,y}B) := D_{x,y}(\kappa_v B).$$

Here u is fresh and unused in t; all substitutions and insertions are capture-free. In a de Bruijn implementation, placing t in the new iterator body must lift its free indices. The asymmetry is useful:

$$\kappa_0(s = t) \text{ is literally reflexive}, \qquad \kappa_{Sz}(s = t) \;\dashv\vdash\; (s = t).$$

The second statement uses iterator computation and identifies the positive gate with the original equation. Under nested D binders, the outside variable v remains free; the gate does not change the number of formula binders or an equation/compound root.

A.2 The four rule families

Use. For an entry $B(y)$ and a target term t, infer $B(t)$ from one minor derivation of $B(0)$. The minor may instead be omitted exactly when the raw substituted base $B(0)$ has the form $D^j(r = r)$ with identical leaf terms, for some $j \geq 0$:

$$\frac{B(y) \in \Delta \qquad \Delta \vdash_H B(0)}{\Delta \vdash_H B(t)}, \qquad \frac{B(y) \in \Delta \qquad B(0) \equiv D^j(r = r)}{\Delta \vdash_H B(t)}. \tag{Use}$$

These are the one-premise and zero-premise cases of one instruction family. The omission test compares the two literal leaf terms and does not inspect their numerical values. It is valid at every binder depth; an arbitrary compound theorem does not qualify.

Eval. From two equations, infer the result of one permitted computation:

$$\frac{\Delta \vdash_H s = t \qquad \Delta \vdash_H b = r}{\Delta \vdash_H r = \Phi(s,t,b)}. \tag{Eval}$$

The partial operation Φ has exactly two cases:

$$\Phi(s,t,b) = \begin{cases} t, & b \equiv I(0,s;u.h), \\ h[t/u], & b \equiv I(Sc,a;u.h) \text{ and } s \equiv I(c,a;u.h). \end{cases}$$

All other shapes are rejected. In particular, no primitive root-joining or arbitrary transport case is included.

Elim.

$$\frac{\Delta \vdash_H D_{x,y}B(x,y) \qquad \Delta, B(b,y) \vdash_H A}{\Delta \vdash_H A}. \tag{Elim}$$

The eigenparameter b is fresh for the conclusion, original context and fixed parameters. The new retained entry is the single template $y \mapsto B(b,y)$, with no separately stored zero instance.

Intro. Define

$$\eta(B,t,z) = \begin{cases} z, & B \text{ is a literal equation and } t \equiv 0, \\ 0, & \text{otherwise.} \end{cases}$$

Then

$$\frac{\Delta, \kappa_v(B(t,\eta(B,t,z))) \vdash_H B(t,Sz)}{\Delta \vdash_H D_{x,y}B(x,y)}. \tag{Intro}$$

The eigenparameter z is fresh for Δ, t and all fixed parameters of B. The displayed new entry is a template in its fresh argument v; z is a free parameter there. The witness-directed choice of cases or progress is exactly the repository's established witness rule.

There are no further primitive rules or nonlogical axioms. In particular, there is no Hyp rule and no Refl rule. The designated contradiction is $0 = S0$.

A.3 Recovering computation and numerical equality

Reflexivity. Use the fixed entry $\rho(y) = (y = y)$ at target t. Its base is literally $0 = 0$, so no minor is required and the conclusion is $t = t$.

Reflexive prefixes. Every well-scoped $D^j(r = r)$ is derivable in every context. Prove this by induction on j, independently of the gate retrieval theorem. The case $j = 0$ is the preceding derived reflexivity. For $j > 0$, apply Intro with witness $1 = S0$, which forces case mode. Ignore its temporary gated assumption. The required successor target has $j - 1$ binders, and its leaf terms remain identical because both undergo the same substitutions. The induction hypothesis supplies that target. Consequently every omitted Use minor can be expanded into an actual derivation using only the fixed initial matrix and the displayed rules.

The two computations. Zero computation is Eval on the reflexivities $a = a$ and $I(0,a;u.h) = I(0,a;u.h)$. Successor computation is Eval on the reflexivities of $I(c,a;u.h)$ and $I(Sc,a;u.h)$.

Root joining. From $s = t$ and $s = r$, put $Z = I(0, s; u.0)$. Eval on $s = r$ and $Z = Z$ gives $Z = r$; Eval on $s = t$ and $Z = r$ gives $r = t$. Symmetry is this derived joining rule with one premise reflexive; transitivity follows by one symmetry and one joining.

Transport in a term template. Suppose $s = t$ and $h(s) = r$. Set

$$Z = I(0, s; u.h(u)), \qquad P = I(S0, s; u.h(u)).$$

Zero-case Eval yields $Z = s$ and $Z = t$. Successor-case Eval on $Z = s$ and $P = P$ gives $P = h(s)$, hence $P = r$ by the preceding derived equality rules. Successor-case Eval on $Z = t$ and $P = r$ now yields $r = h(t)$. This is the native kernel's crossed one-template transport rule. Consequently all numerical equality reasoning used below is derived inside the four-family calculus.

A.4 The gate retrieval theorem

For every formula A and every context containing ρ:

1. $\Delta \vdash_H \kappa_0 A$.
2. A derivation of $\kappa_1 A$ can be converted effectively to a derivation of A, and conversely.
3. If $\kappa_v A$ is a retained context entry, then $\Delta \vdash_H A$.

The transformations are uniform under enlarging the context and under renaming eigenparameters.

Proof of assertion 1. The formula $\kappa_0 A$ has the same D prefix as A and literally identical iterator terms at its leaf. It is therefore an instance of the independently established reflexive-prefix theorem. No retrieval lemma is used in this step.

Simultaneous proof of assertions 2 and 3. At an equation, assertion 2 is computation and equational transport. Suppose assertion 3 has been proved for all formulas with fewer binders than $A = D_{x,y}B$.

To convert $\kappa_1 A = D_{x,y}\kappa_1 B$ to A, eliminate the source at a fresh b, retaining the matrix $\kappa_1 B(b, y)$. Introduce the target with witness b. Because b is a variable rather than the literal zero, this is a case introduction. Its local entry is $\kappa_v(B(b, 0))$. Assertion 3 at the smaller formula depth supplies $B(b, 0)$. Assertion 2 at that smaller depth converts it to $\kappa_1 B(b, 0)$. Use the retained source matrix at target Sz and convert its result $\kappa_1 B(b, Sz)$ back to $B(b, Sz)$. Close Intro and Elim.

The reverse conversion starts by eliminating $D_{x,y}B$ and introducing $D_{x,y}\kappa_1 B$ with the same fresh witness b. Retrieve its local zero assumption by assertion 3 at the smaller depth, convert to the ungated zero instance, Use the retained $B(b, y)$ matrix at Sz, and convert back to the gated successor instance. Thus assertion 2 at the current depth uses assertion 3 only at a strictly smaller depth.

Finally, prove assertion 3 at the current depth. Use the retained entry $\kappa_v A$ at target 1. Its zero instance is leaf-reflexive at every formula depth, so the minor is omitted by the explicitly stated Use rule. The result is $\kappa_1 A$, which assertion 2 converts to A. This completes a well-founded simultaneous induction. If desired, the omitted minor can be expanded by assertion 1; no unproved compound assumption has been discarded.

A.5 Soundness in the original ordinary realization

Read context entries as their conditional closures in T_{eq}. Formula induction in ordinary intuitionistic logic with equality proves

$$\kappa_0 A, \qquad \kappa_{Sz} A \leftrightarrow A.$$

At atoms the latter uses the two computations, and at a D binder it uses the ordinary extensionality of the displayed existential conditional definition. Numerical coverage therefore proves

$$A \;\longrightarrow\; Q_v \kappa_v A. \tag{Gate introduction}$$

Indeed, under A, a numerical argument is zero or a successor; the first gated instance is a theorem, and the second is equivalent to A.

Induct now on a four-family derivation. Eval is a valid equational consequence. Use is conditional-universal specialization; when its minor is omitted, the reflexive-prefix theorem supplies exactly that missing minor. Elim is ordinary existential elimination. For Intro, its ordinary temporary fact $B(t, \eta)$ supplies the retained encoded assumption by Gate introduction. When $\eta = 0$, the successor premise and numerical coverage establish the conditional universal conclusion. When $\eta = z$, the matrix is an equation, and equation induction with retained side assumptions establishes the same conditional conclusion. Existential introduction at t completes the argument. The initial ρ closure is an equality theorem. Hence the calculus is sound under the unchanged ordinary reading.

A.6 Completeness and the interpretation composites

Let W be the pinned seven-rule witness calculus `WPrf`. The repository proves `wprf_iff_nprf`; its introduction uses the same witness test as above. Translate its formulas by fixing equations and D and putting

$$(Q_y B)^\circ = D_{r,y} B^\circ$$

with r fresh and unused. The translation preserves root class, fixed parameters and capture-free numerical substitution.

The precise derivation induction is: if $\Gamma \vdash_W A$, and every $C \in \Gamma$ has a proof of C° from a homogeneous context Δ, then $\Delta \vdash_H A^\circ$. Hyp uses the supplied proof. The computations and transport use the equational derivations above. For Q elimination, eliminate the translated dummy-witness D and Use its retained matrix at the original target. For Intro, the gate retrieval theorem supplies precisely the translated local fact from the new entry $\kappa_v(B^\circ(t, \eta))$, so the induction hypothesis applies to the source premise.

For source D elimination, the newly retained matrix is $B^\circ(b, y)$. It supplies the translated source assumption $(Q_y B(b, y))^\circ = D_{r,y} B^\circ(b, y)$ as follows: introduce this dummy-witness binder at witness 1, ensuring case mode, retrieve its local zero fact through the gate theorem, and Use the retained matrix at Sz. Thus the derivation induction also applies to the source elimination premise. This strengthened context statement avoids any unproved primitive cut or clause-construction rule.

For an explicit source context, take Δ to consist of ρ and the entries $\kappa_v(C^\circ)$ for $C \in \Gamma$; gate retrieval provides all needed hypothesis proofs. Hence every source derivation has an effective four-family derivation.

Conversely, the soundness proof translates every new rule to the original ordinary scalar theory. The pinned native completeness theorem returns it to the native calculus. For context bookkeeping, the reading of a matrix entry is exactly its native Q formula, and $Q_v \kappa_v A \leftrightarrow A$ follows from Gate introduction and specialization at 1. Thus the encoded-context composites are also provably equivalent, not merely equiconsistent.

On formulas, the H-to-native translation is the inclusion of the Eq/D language. The other translation only replaces Q by D with an unused witness. The native theory proves this replacement equivalent in both directions, so formula induction gives both round trips. The H-side formula composite is literally the identity. Both term maps and both numerical domains are the identity; numerical equality remains actual equality. Composing with the repository's native-to-iPRA bi-interpretation gives the requested bi-interpretability, both numerical composite graphs $x = y$, and equiconsistency. The designated contradiction is fixed. The arithmetic bridge and its existing proof-assistant verification are inherited results; the new gate and context theorem is a written proof unless separately formalized.

A.7 A flat formula representation

The Eq/D grammar is unary above its atomic leaf. Every formula is uniquely of the form

$$D_{x_1,y_1} \cdots D_{x_k,y_k}(s = t).$$

It may therefore be represented by (k, s, t) instead of a recursively nested formula tree, with the terms scoped under the $2k$ displayed bound variables. A gate leaves k unchanged and transforms the single leaf equation; a binder adds one to k. Under a relative-index representation, the gate's external variable must be placed outside all $2k$ formula-bound variables, and placing the old right-hand term under the iterator binder requires lifting its exterior positive indices. In the mixed convention of the delivered checker, negative free-parameter levels remain unchanged. This is a representation theorem with no change of language or interpretation. Checker measurements must still include the integer and scope validation, capture-free substitutions and gate construction.

Scope of the simplicity claim. The mathematical deletion is the separate antecedent field of every retained clause. Fact assumptions and conditional closures now use one homogeneous context grammar. The omitted-minor reflexivity test and the gate are explicit parts of the trusted checker. Four instruction families do not beat the repository's existing four-family count by themselves; any software-size improvement must be the result of a matched, complete, measured implementation.

Implementation evidence. The separate homogeneous implementation was checked on 400 generated formulas for its literally reflexive zero gate and on 800 positive-control instances against environment-based numerical evaluation. Six closed certificates passed, including equation induction whose local hypothesis is recovered from its gated context entry, followed by specialization. These finite checks are reproduced by `code/audit_homogeneous.py`; they are separate from the primary checker's subprocess comparison.

References

[1] Vladimir Reshetnikov, *Concise presentations of intuitionistic primitive recursive arithmetic*, pinned repository README, commit `17898ea7e6a1300dfdcad71040d5daeed67916d4`. https://github.com/VladimirReshetnikov/ProveIt/blob/17898ea7e6a1300dfdcad71040d5daeed67916d4/Logic/PrimitiveRecursiveArithmetic/ConciseIPRA/README.md.

[2] Same repository, `Lean/ConciseIPRA/NativeClausal.lean`, including `CPrf`, `cprf_iff_nprf`, and the clause translations.

[3] Same repository, `Lean/ConciseIPRA/NativeUniformClauses.lean` and `Research/uniform_clauses/code/checker.py`; the four-family clause interface and its original standalone implementation.

[4] Same repository, `Lean/ConciseIPRA/NativeEval.lean` and `Research/clausal_two_case/`; computational equality and its two-case reduction.

[5] Same repository, `Lean/ConciseIPRA/NativeOneIntro.lean`; the exact equivalence of the witness-directed single introduction and the native kernel.

[6] Same repository, `Lean/ConciseIPRA/NativeBiInterpretation.lean`, `NativeRoundTrip.lean`, and `BiInterpretation.lean`; ordinary readings, native round trips, and the scalar–iPRA bi-interpretation.

[7] Same repository, `Research/certified_zero/reference/`: the inherited choice basis, ordinary bridge, scalar predecessor, equality reflection, and primitive recursion proofs. These five source files are included in this package with provenance and the repository's MIT-0 license.